Advanced Labs II

ALII-cd: Control System Design Report

Control Systems Design Project Report

Andrea Trevanna 202318030

1. Objective of the project

In this project, we worked on understanding PID control and learning to use MATLAB and Simulink. The PID control is not only used for simulation. We further learn how to implement the PID controller we designed to control an actual physical system, DC motor. For PID control, we use different inputs (step and trapezoid system). For our project, we tune different values of PID gains (the proportional gain K_p, the integral gain K_i, and the derivative gain K_d) to see different responses, stabilise the response, and prevent overshoot.

2. Principle

Figure 1 shows a general system response for a system. The x-axis is time and the y axis is the angular velocity of the motor in this system.

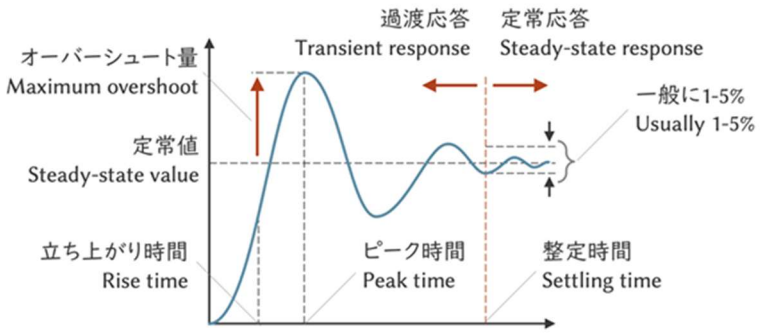


Figure 1 General System Response

We calculated the transfer function for this PID control system to create our own model that fits this system before we designed the controllers. The first-order + dead time system that we calculated to model this motor can be expressed by the following transfer function,

$$\overline{G}_m(s) = \frac{k}{\tau s + 1} e^{-Ls}$$

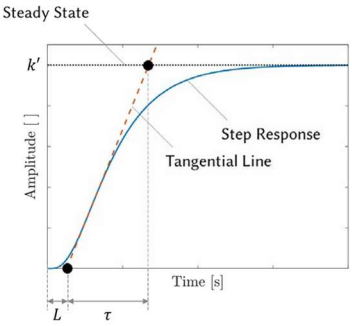


Figure 2 Simple system identification method using step responses

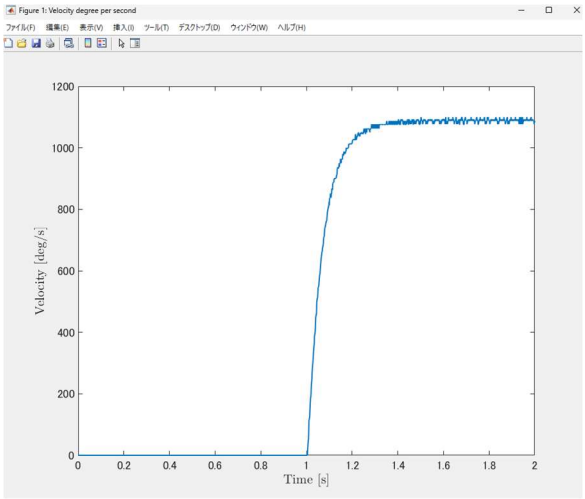


Figure 3 Velocity degree per second graph to identify system using step response

. To determine k, τ, and L, we use the system identification system using step response as in the Advanced Labs II Textbook AlI-cd: Control System Design weeks 3 equations (3.6)-(3.13) with t1= 1.01, t2=1.01, L=0.01 as per Figures 2 and 3. Thus, k=4.3137, τ=0.09, L=0.01≈0Therefore, the system can be represented as follows,

$$\overline{G_m}(s) = \frac{k}{0.09s + 1}$$

This is what is entered in the first-order + dead time part of the PID controller we designed.

PID control stands for Proportional-Integral-Derivative control. It is a widely used feedback control technique in systems that require precise and stable performance. The controller adjusts the output based on three components: proportional, integral, and derivative.

a) Proportional (P): The proportional component responds to the current error between the desired setpoint and the actual system output. It applies a correction that is directly proportional to the error. A larger error results in a stronger corrective action. However, if the proportional gain is too high, it may cause the system to overshoot or oscillate.

b) Integral (I): The integral component considers the accumulated error over time. It sums up past errors to eliminate any steady-state bias or offset that the proportional term alone cannot correct. This term helps drive the error to zero but can also cause overshooting or instability if the integral gain is set too high.
c) Derivative (D): The derivative component predicts future error by measuring the rate of change of the error. It provides a damping effect that helps reduce overshoot and smoothens the system response. While it improves stability, excessive derivative gain can amplify noise and lead to erratic control behaviour.

3. Explanation for the project

The PID control system we designed to manage the response of a motor system connected to an Arduino Due 1 is in Figure 4. This program is a simulation of the actual response, based on a transfer-system model that was calculated in week 3. It consists of two parts: controller and plant.

Controller: Firstly, this system consists of an input. The figure shows a step signal but two different inputs: a step signal and a trapezoidal signal. The PWM saturation range was set between 0 and 255. The Error1 scope is used to monitor the quality of the signal before it passes through the PID controller. After that, it passes through a proportional gain, an integral gain and a derivative gain . We change different values for each gain to see the response. A random disturbance (shown in the figure) and a step disturbance was added to simulate real-life conditions. The simulation time was set to 7 seconds. Plant: The plant consists of the main transfer function. After adding a dead time + first order that was calculated using the transfer function shown in section 2, the output is taken. The control output corresponds to the angular velocity set at a target value of 600 deg/s.

Plant: The plant consists of the main transfer function. After adding a dead time + first order that was calculated using the transfer function shown in section 2, the output is taken. The control output corresponds to the angular velocity set at a target value of 600 deg/s.

The PID model used for week 4 project is based on the PID model created on week 3 in Figure 4.

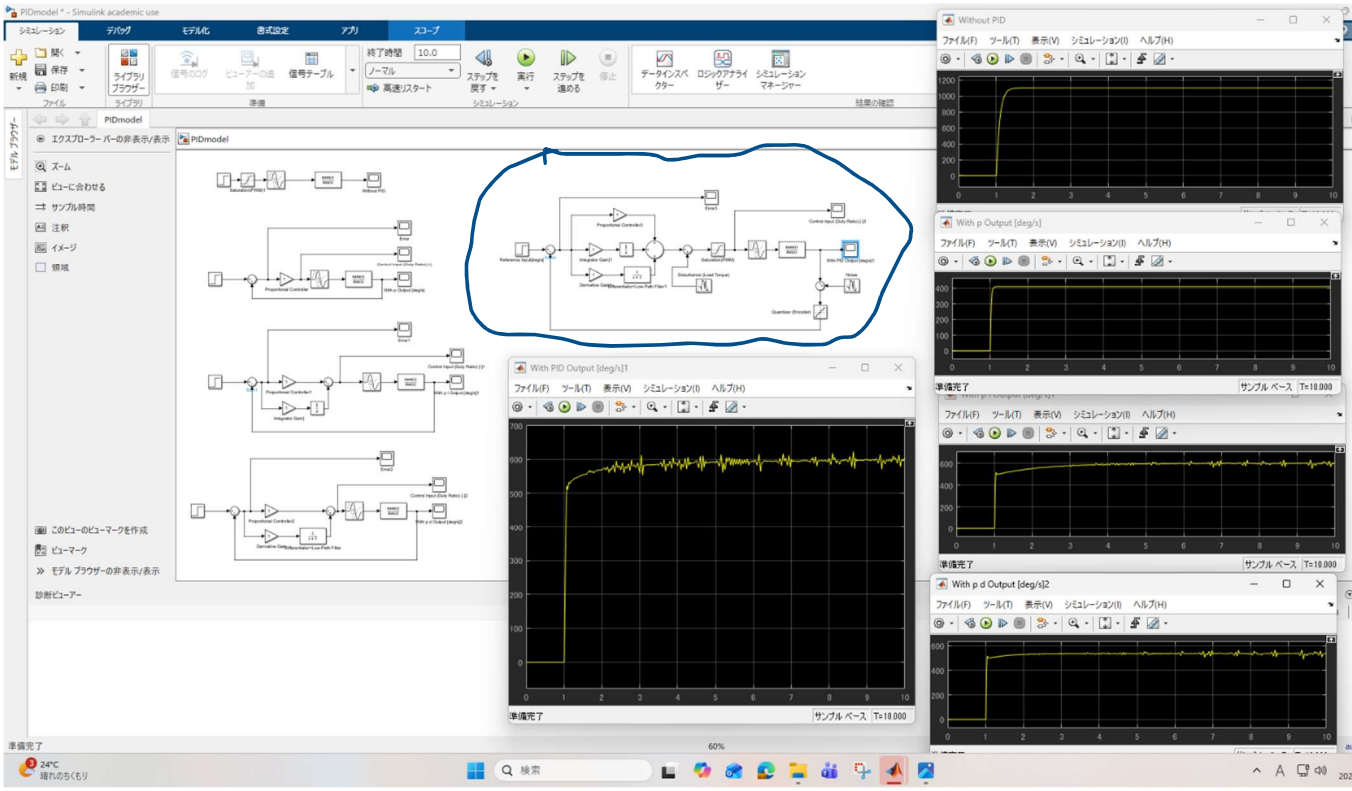


Figure 4 Precursor for the PID design in project

The PID model itself is in Figure 5.

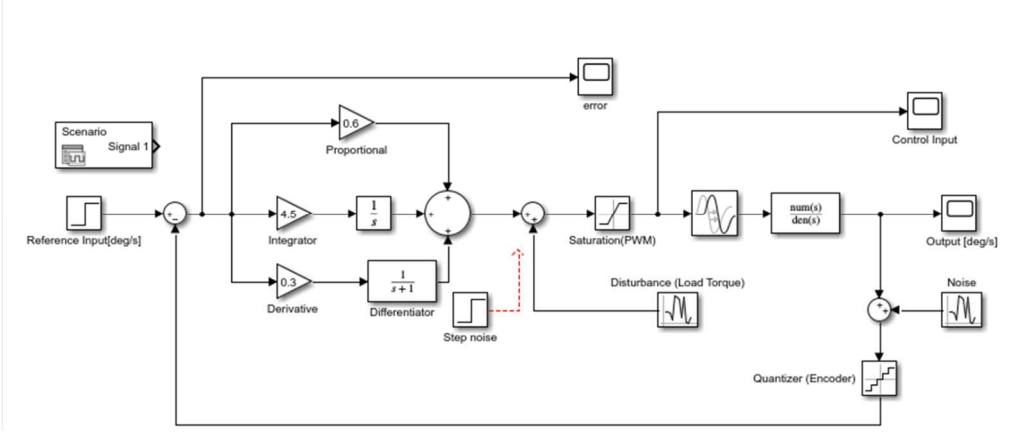


Figure 5 PID control design(simulation)

The PID controller has an input of step signal or trapezoid signal (trapezoid signal is represented by Signal 1), and the PID controller is connected to error block, disturbance, the transfer function block of the model, noise and encoder.

The PID control system connected to the actual physical system DC motor is in Figure 6. This does

not include any random noise as the real motor moves which automatically introduces noise due to component resistances. The output of the signal is the velocity of the motor.

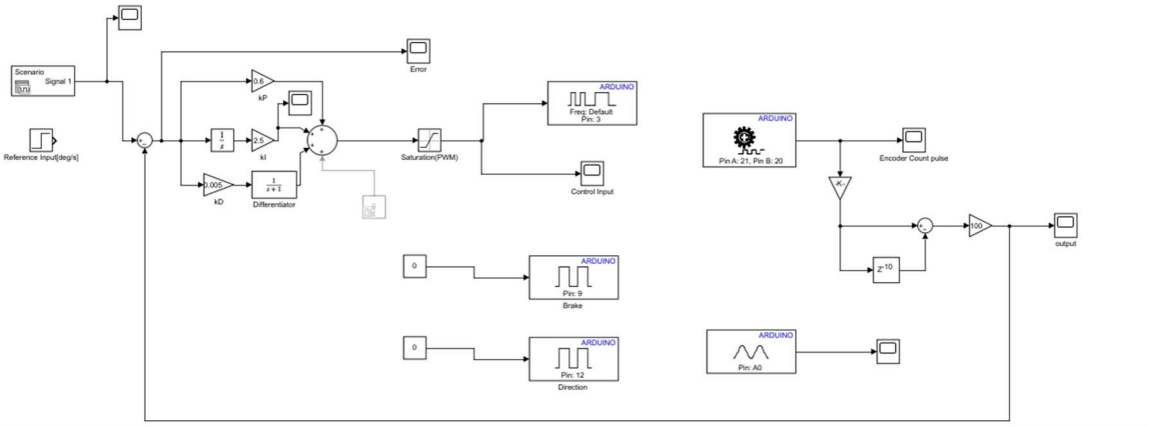
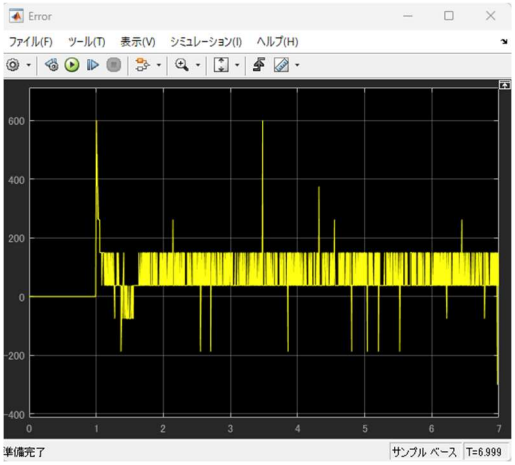


Figure 6　PID control design (real)

The error has a random signal



4 Experimental results

Through trial and error, we have tried to find optimum values for the proportional gain K_p, the integral gain K_i, and the derivative gain K_d through trial and error. This has resulted in many trials.

1. Simulation

   For the simulation, we choose Kp=0.6, Ki=4.5, and Kd=0.3. This is based on our many trials of determining the optimal values of Kp, Ki and Kd through the PID controller connected to DC motor in Part 2 of this report.　We modify the output response by connecting the step input/trapezoid input PID controller , which is then fed into control input and disturbance labelled noise. For the disturbance, we use either random or step. We obtained the results below.

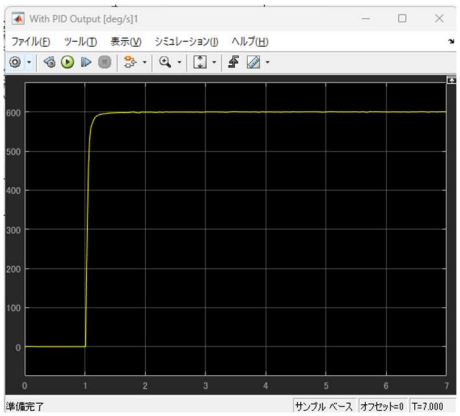


Figure 7 Step input to random noise

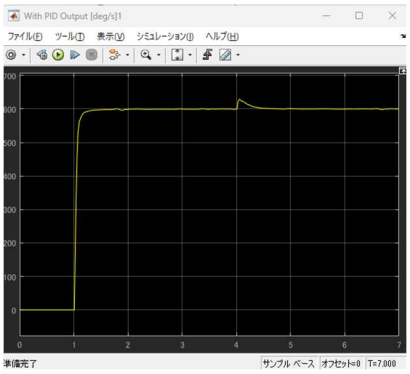


Figure 8 Step input to step noise

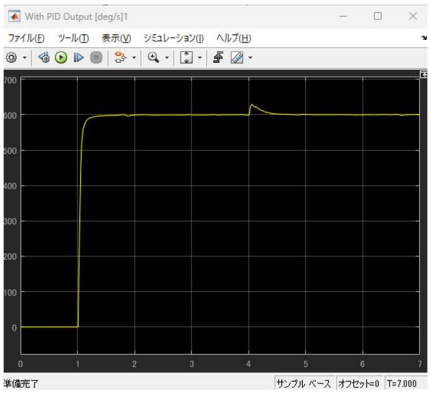


Figure 9 Trapezoid input to random noise

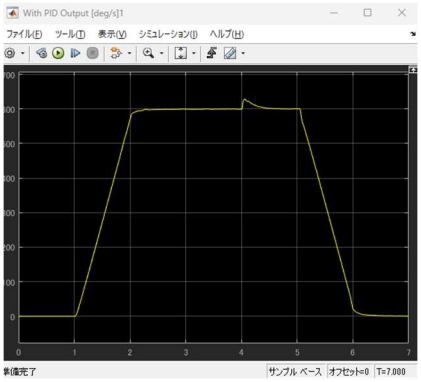


Figure 10 Trapezoid input to step disturbance

2. Real motor input

a) Step input: Determining optimal Kp, Ki and Kd gain values through trial

First, we tried only using the proportional gain Kp while setting Ki and Kd both to zero. . Based on the following results, it seems that the system is most stable at Kp= 0.6 or Kp=0.7. When Kp has a larger value or a value that is too small, the output response begins to fluctuate and becomes more unstable.

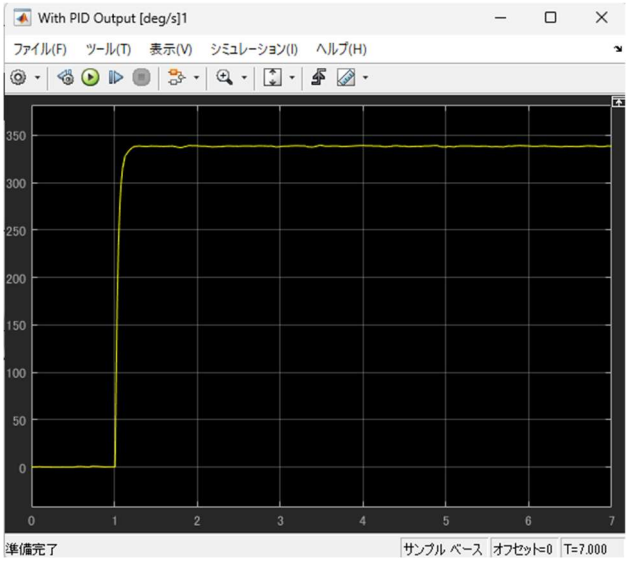


Figure 11 Step response at Kp=0.3, Ki=0, Kd=0

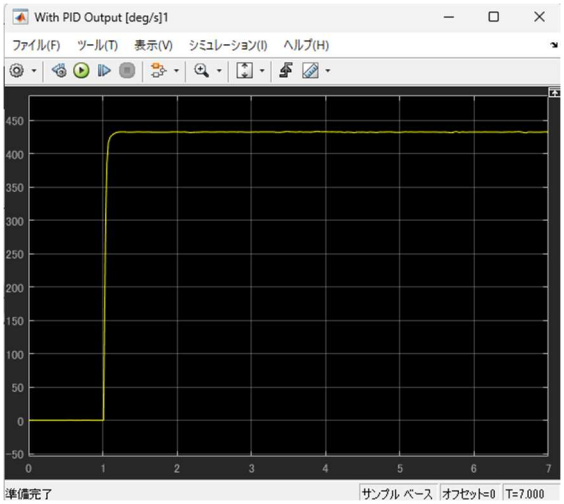


Figure 12 Step input with Kp=0.6, Ki=0, Kd=0

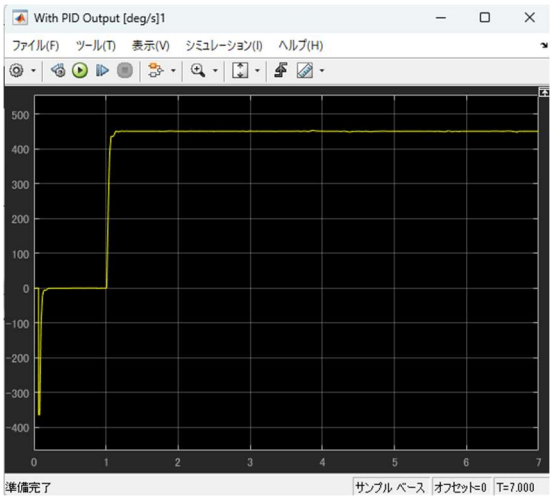


Figure 13 Step input, Kp=0.7, Ki=0, Kd=0

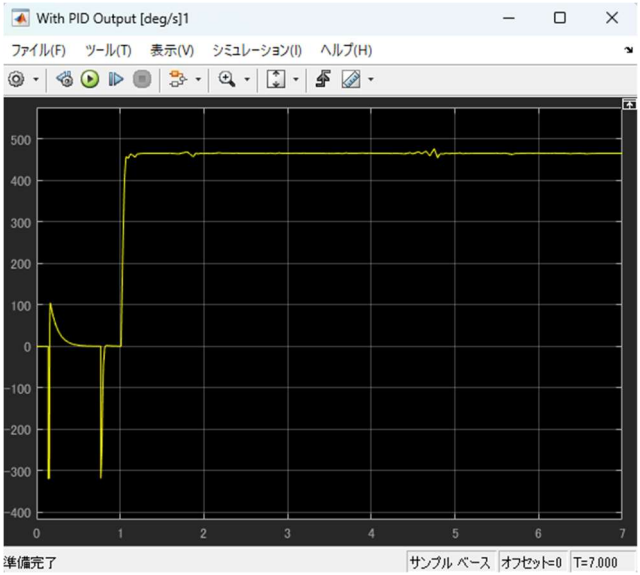


Figure 14 Step input, Kp=0.8, Ki=0, Kd=0

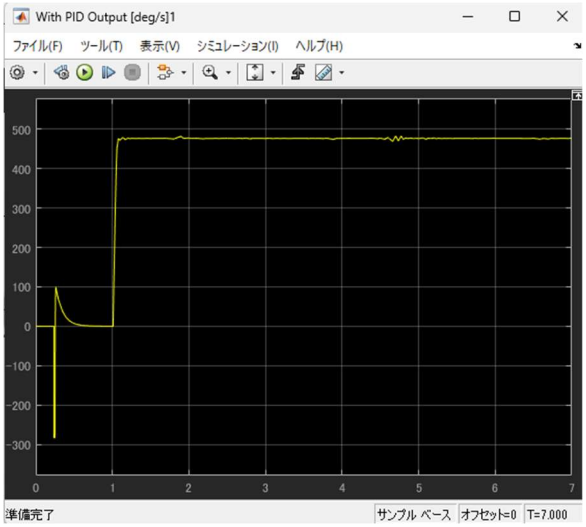


Figure 15 Step input, Kp=0.9, Ki=0, Kd=0

Next, we try Kp=0.6 with different values of Ki, while setting Kd=0. The results are in Figures 16-19. The response shows undershoot and then steeper increase at lower values of Ki, while showing instability and/or overshoot at higher values of Ki. Based on the results, the response shows less undershoot/overshoot at Ki=4.0-5.0. Overall, the optimal value for Ki seems to be 4.0, with almost no over/undershoot and no noise in the response signal.

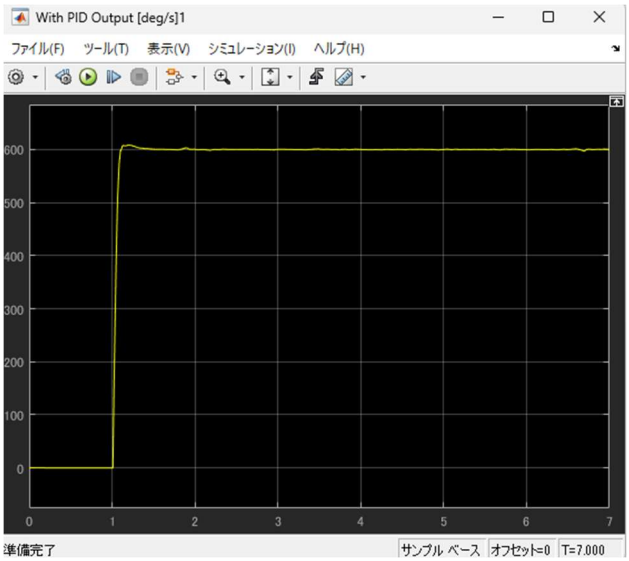


Figure 16 Step input with Kp=0.6, Ki=6,Kd=0

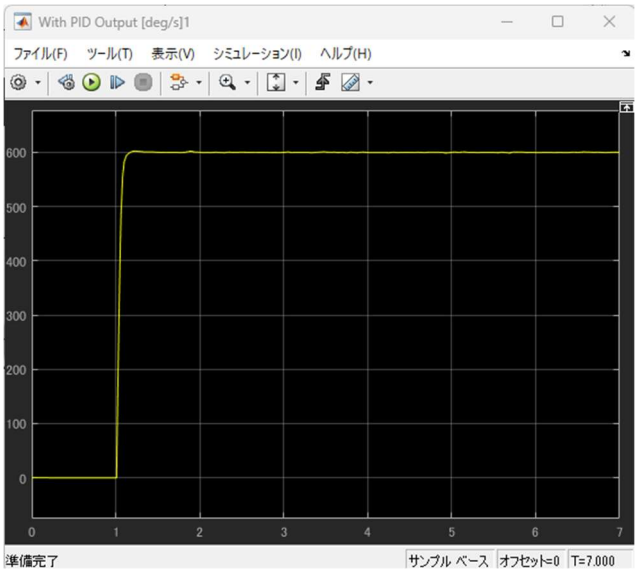


Figure 17Step input with Kp=0.6, Ki= 5 .5, Kd=0

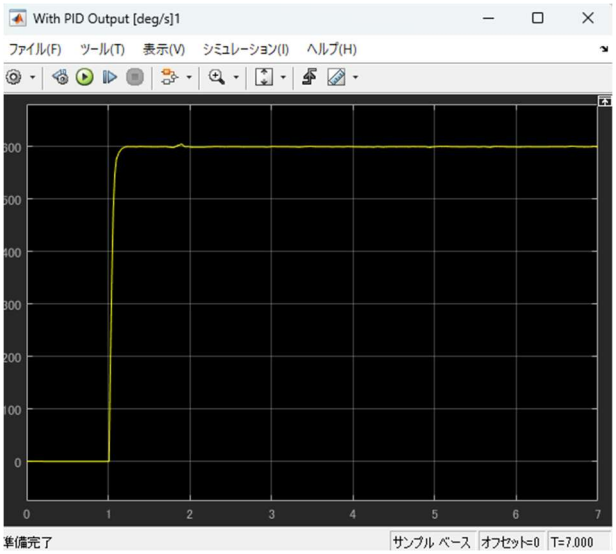


Figure 18 Step input with Kp=0.6, Ki=5.3, Kd=0

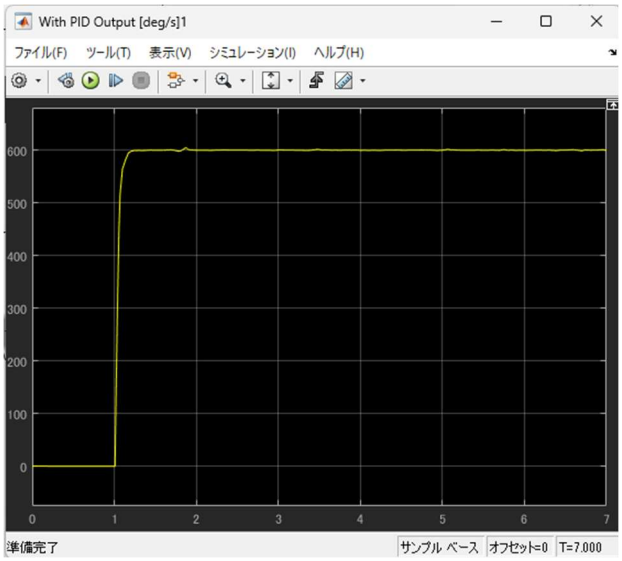


Figure 19 Step input with Kp=0.6,Ki=5.1,Kd=0

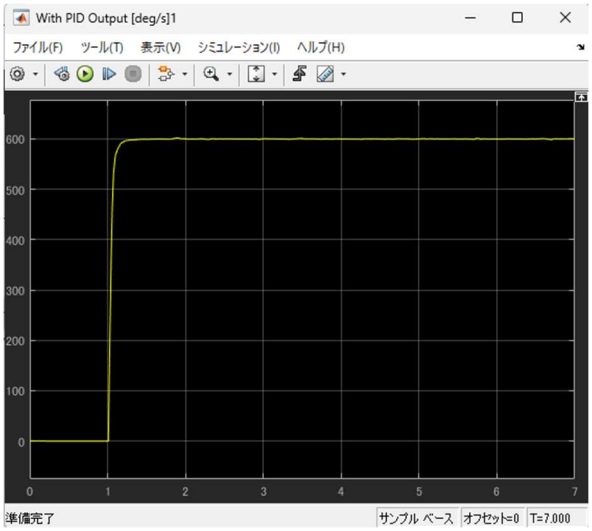


Figure 20 Step input with Kp=0.6, Ki=5.0, Kd=0

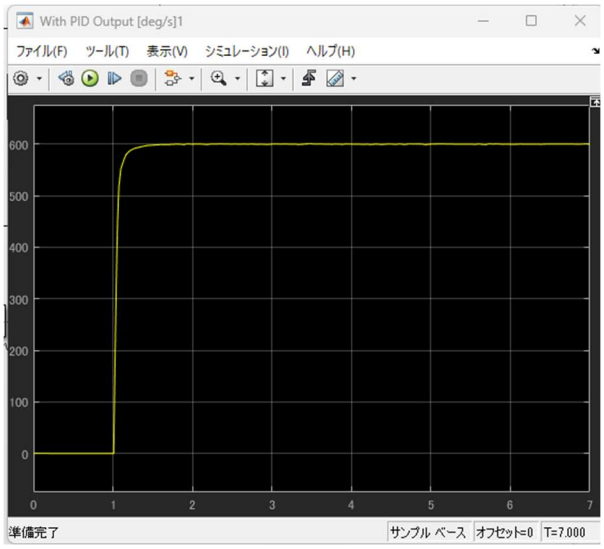


Figure 21 Step input with Kp=0.6, Ki=4.5, Kd=0

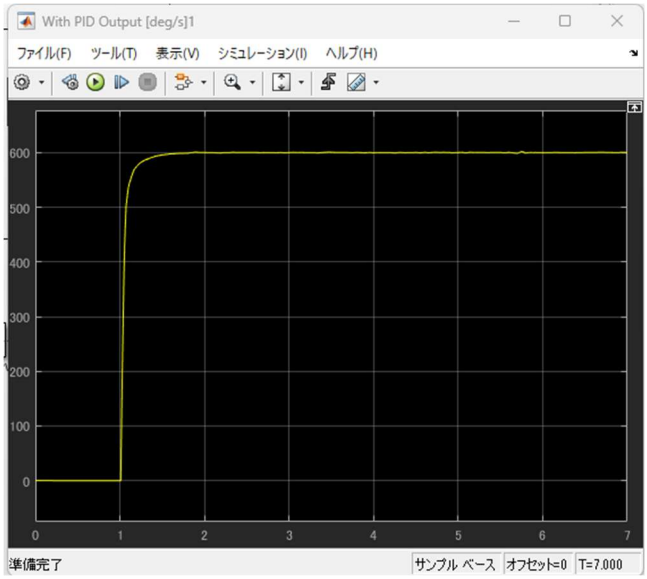


Figure 22 Step input with Kp=0.6, Ki=4, Kd=0

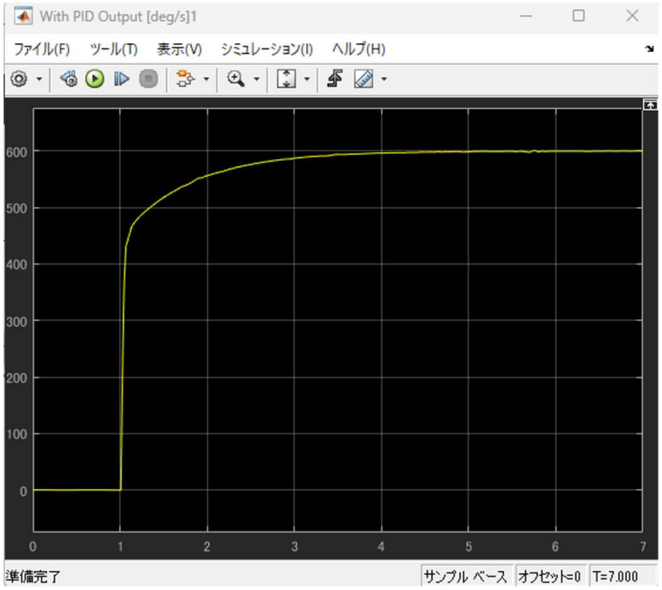


Figure 23 Step input with Kp=0.6, Ki=1, Kd=0

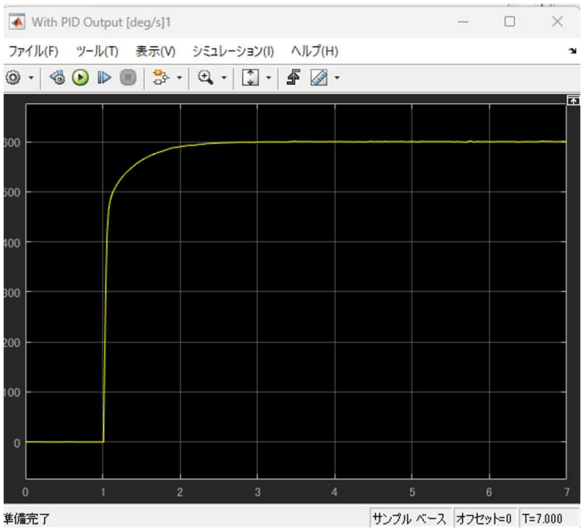


Figure 24 Step input with Kp=0.6, Ki=2.0, Kd=0

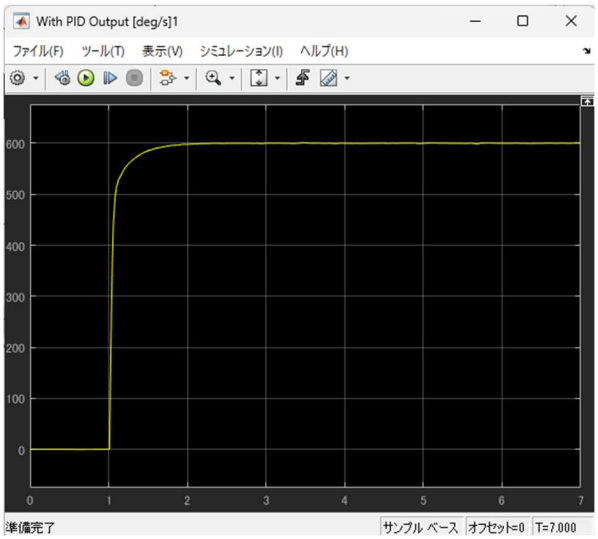


Figure 25 Step input with Kp=0.1, Ki=3.0, Kd=0.

Next, after determining “optimal” Kp=0.6 and Ki=4.5 values, we seek to tune the optimal Kd gain value of our PID controller, also through trial of substituting different Kd values. From this, we can gain two conclusions: 1) a Kd value that is too large can lead to overshoot and instability 2) With our Kp=0.6 and Ki=4.5 value, having Kd=0 leads to more stable response compared to having Kd. Having Kd can induce small ripples in the response. Compare the results with Figure 21.

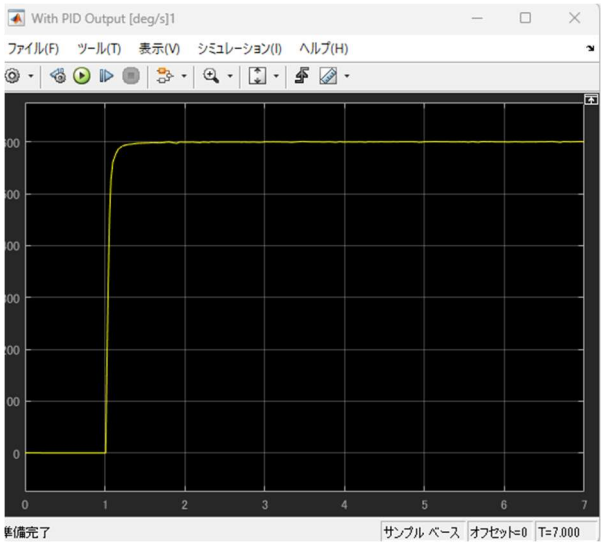


Figure 26 Step input, Kp=0.6,Ki=4.5,Kd=0.3

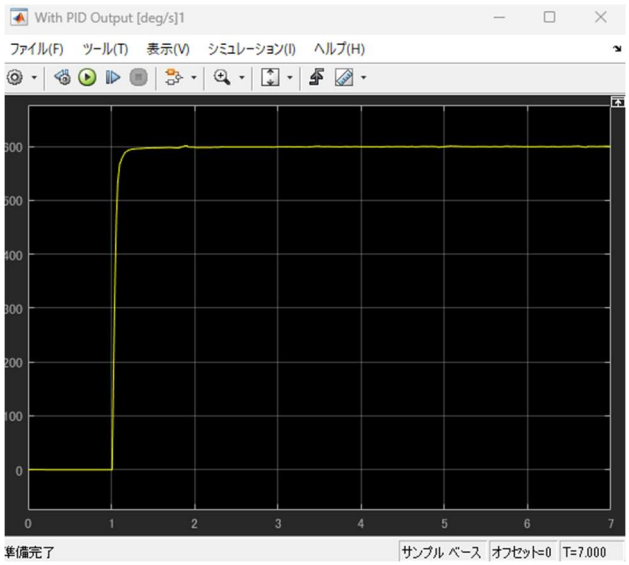


Figure 27 Step input, Kp=0.6,Ki=4.5,Kd=0.5

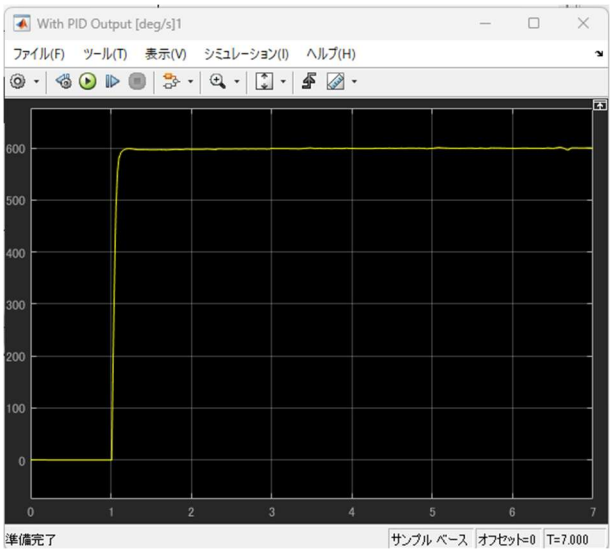


Figure 28 Step input, Kp=0.6, Ki=4.5, Kd=1

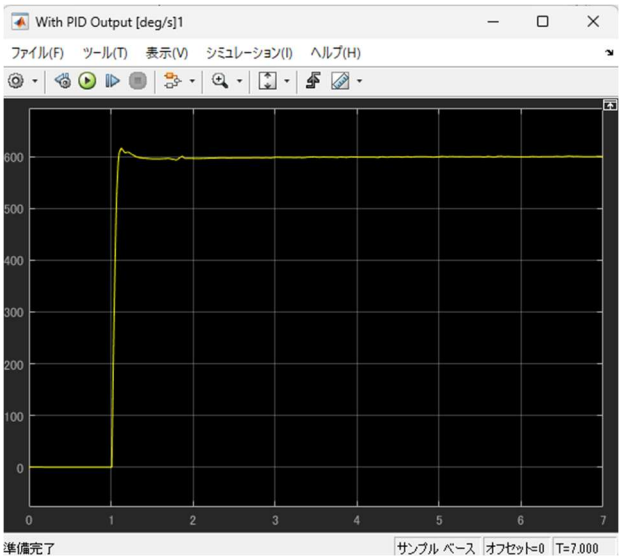


Figure 29 Step input Kp=0.6, Ki=4.5, Kd=2

We also tried Kp, Ki and Kd values different from our optimal Kp and Kd values, which expectedly lead to less than optimal, much more unstable output results.

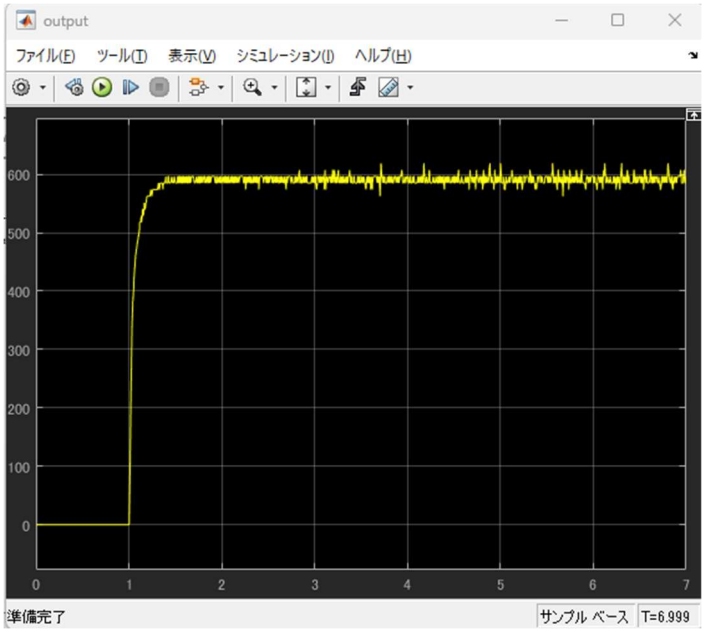


Figure 30 Kp:0.6 Ki:2.5 Kd:0.05

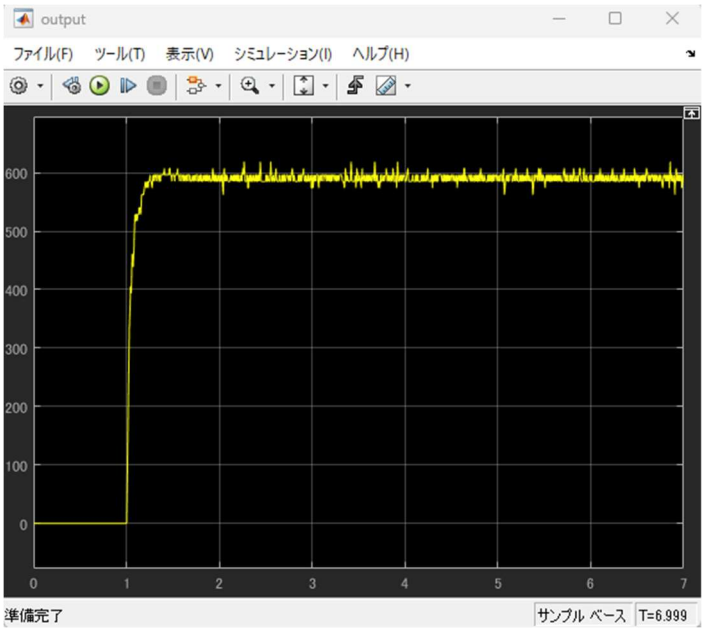


Figure 31 Kp:0.6 Ki:3 Kd:0.05

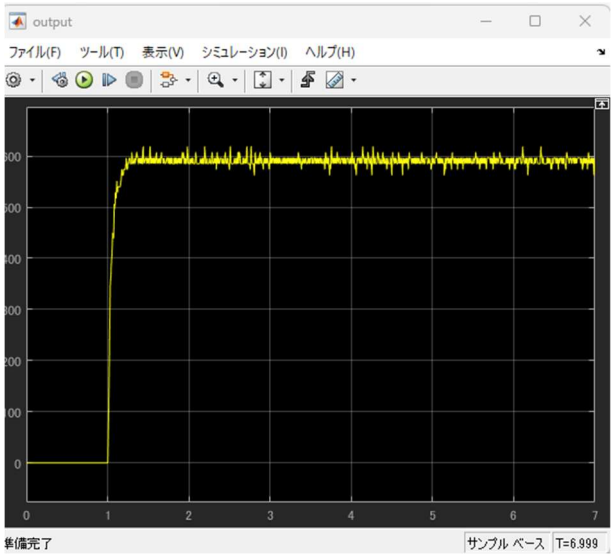


Figure 32 Kp:0.6 Ki:3 Kd:0.07

b) Trapezoid input: Determining optimal Kp, Ki and Kd gain values through trial

First, setting Ki=0.

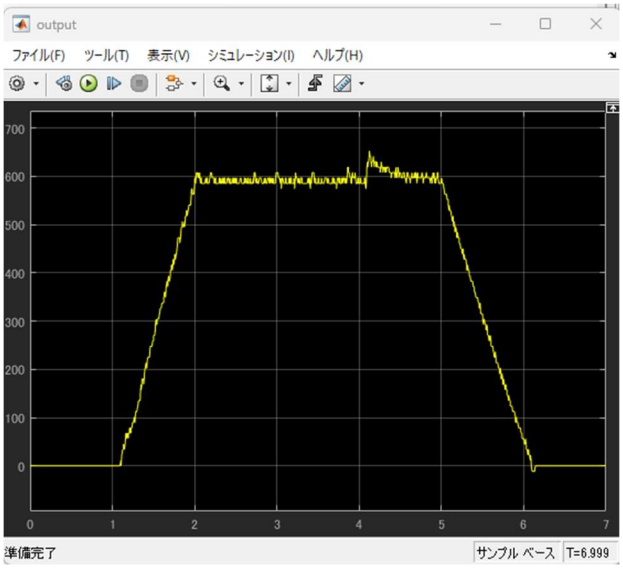


Figure 33 Trapezoid input, Kp=0.62, Ki=0, Kd=0.05

Then, varying Ki gain values. The output response shows most stability and resemblance to an ideal trapezoid signal at Kp=0.62, Ki=5, and Kd=0.05.

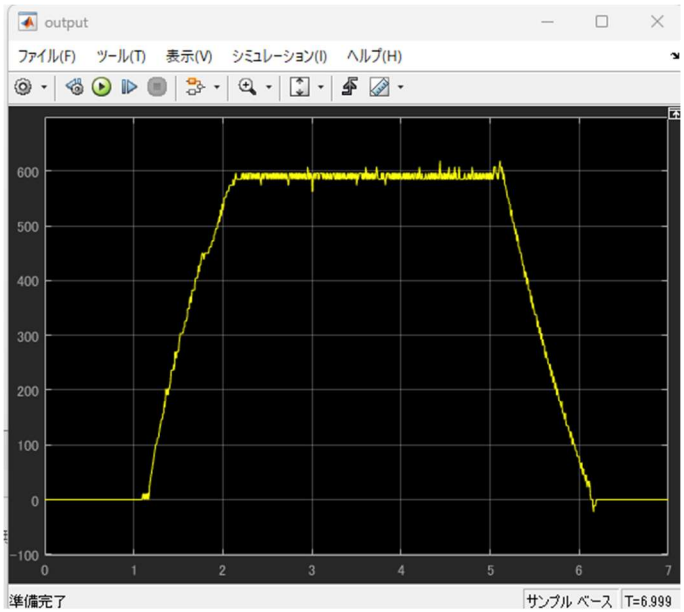


Figure 34 Trapezoid input, Kp=0.62,Ki=3,Kd=0.05

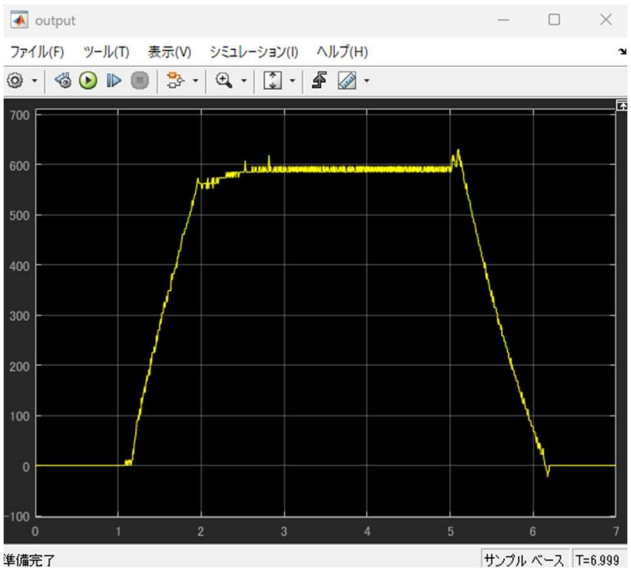


Figure 35 Trapezoid input Kp=0.62, Ki=4, Kd=0.05

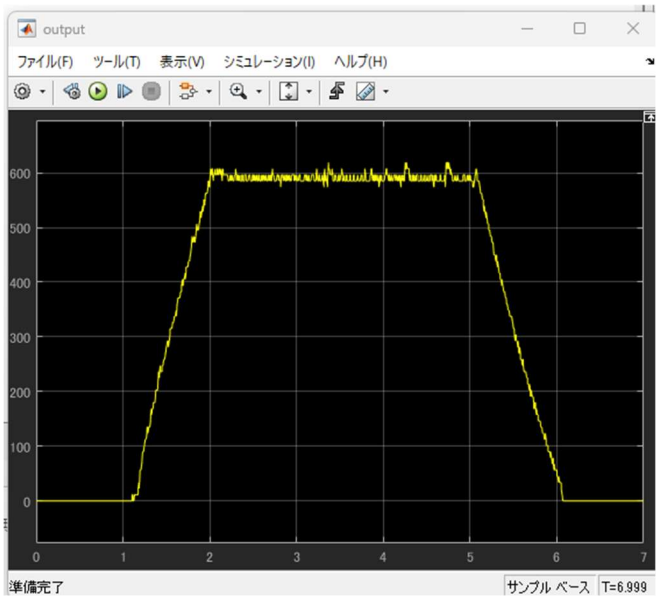


Figure 36 Trapezoid signal input Kp=0.62, Ki=5, Kd=0.05

Discussion

Here is an explanation for what gain values were chosen and why. The gain values for each controller were chosen based on the effect of each controller as explained in section 2 and the analysis of the signal with no control.

The values for Ki, Kp and Kd are decided from trial and error many times. Thus, we can see the general trend.

The role of each Ki, Kp and Kd gain:

1. Increasing the proportional gain (Kp) has the effect of proportionally increasing the control signal for the same level of error. The fact that the controller will "push" harder for a given level of error tends to cause the closed-loop system to react more quickly, but also to overshoot more. Increasing the proportional gain helps the signal reach the target value sooner. However, when Kp is too large, it leads to increased overshoot. Another effect of increasing Kp is that it tends to reduce, but not eliminate, the steady-state error. Thus, a balance of Kp is important. The optimal value is at Kp=0.6.

2 . The addition of a derivative term to the controller (Kd) adds the ability of the controller to "anticipate" error. With simple proportional control, if Kp is fixed, the only way that the control will increase is if the error increases. With derivative control, the control signal can become large if the error begins sloping upward, even while the magnitude of the error is still relatively small. This anticipation tends to add damping to the system, thereby decreasing overshoot. The addition of a derivative term, however, has no effect on the steady-state error. Increasing the derivative gain made the system sensitive to noise, not allowing the system to reach a steady state. This may explain why for the step input, the output is more stable when only Kp=0.6 and Ki=4.5 is used with Kd=0 compared to using Kd. Therefore, this value Kd at later testing was kept low to 0.05 for the trapezoid input, which is tested later than the step input.

2. The addition of an integral term to the controller (Ki) tends to help reduce steady-state error. If there is a persistent, steady error, the integrator builds and builds, thereby increasing the control signal

and driving the error down. Increasing the integrator value helped stabilize the system by reducing steady-state error. Although normally this slows the oscillations, in this case, it created a steady-state sooner, decreasing the settling time without affecting other parameters. A drawback of the integral term, however, is that it can make the system more sluggish (and oscillatory) since when the error signal changes sign, it may take a while for the integrator to "unwind".

6. Conclusion

In this lab, we designed and tuned PID controllers with the goal of reducing the settling time of a motor control system. Using both MATLAB/Simulink simulations and a physical setup, we tested step and trapezoidal inputs and adjusted the gain values based on the system's response. For the step and trapezoidal inputs, we test optimal values of the PID gains Kp, Ki, and Kd through trial and error to investigate the effect on the system response. Kp leads to increased target value reaching speed, however when Kp is too large it ca lead to an overshoot. Ki helps to stabilise the system, however when Ki is too little, it can lead to initial undershoot and then steeper increase. Kd, on the other hand, can introduce more noise to the system, so a small value of Kd may be ideal. The balance between correct Kp, Ki and Kd gains is a tricky ordeal, but affects significantly for a PID-controlled system. The optimal gain values found for the step input is Kp=0.6, Ki=4.0 or 4.5 and Kd=0; while for the trapezoid input Kd=0.62, Kp=5, and Kd=0.05.

7. Impression

These 4 weeks of control system are an intensive continuation into application of basic control system design including frequency response and PID control through MATLAB and Simulink. We also learn how to control an actual physical system through Simulink connected to Arduino. The exercises were theoretical and challenging, but helped to sharpen mental calculation, modelling, and sketch possible response and stability of a system, which is very useful to evaluate a system before testing. The appeal of control systems is that with modelling and mathematics, it can be applied to various systems everywhere.

Thank you very much for this class.